

Test Process and Test Plan Guide

Document ID: GKB-TEST-PROCESS-001

Version: 1.0

Scope: General Knowledge Base

Intended use: reusable guidance for test planning and test documentation

Owner: Quality Engineering Office

1. Purpose

This guide defines a practical structure for planning, designing, executing, and reporting tests in engineering projects.

It is intended to help AI-assisted test planning tools produce outputs that are consistent, reviewable, and useful to human test engineers.

The guide is inspired by common test-process standards and professional test documentation practices, especially the ISO/IEC/IEEE 29119 family, which defines concepts, test processes, and test documentation templates for software and systems testing.

2. Core Principles

A useful test plan should be:

- traceable to requirements or risk sources;
- understandable by engineers and non-specialist reviewers;
- executable by a test team;
- realistic with respect to time, resources, tools, and environments;
- explicit about scope and out-of-scope items;
- explicit about evidence expected from execution;
- maintainable when requirements change.

3. Recommended Test Planning Workflow

3.1 Intake

Collect the documents and inputs required for test planning:

- product requirements;
- system architecture;
- interface specifications;
- safety or security requirements;
- operational constraints;
- known incidents and defects;
- regulatory or customer obligations;
- previous test reports.

The test planner should identify which documents are authoritative. If two documents conflict, the conflict should be recorded rather than silently resolved.

3.2 Requirement and Risk Review

Before designing tests, review the requirement set for:

- missing acceptance criteria;
- vague wording;
- non-verifiable statements;
- duplicate requirements;
- conflicting constraints;
- high-risk behaviors;

- safety, security, or reliability implications.

If the requirement cannot be tested, the test plan should state what clarification is required.

3.3 Test Strategy Selection

Select the test strategy according to requirement type.

Requirement type	Typical test strategy
Functional	Scenario, API, workflow, system test
Performance	Load, stress, timing, resource monitoring
Reliability	Endurance, fault injection, recovery test
Security	Authentication, authorization, abuse case, vulnerability test
Interface	Contract, compatibility, boundary, protocol test
Safety	Hazard-based, failure-mode, acceptance evidence test
Usability	Task-based observation and user acceptance

High-risk requirements should receive deeper test coverage than low-risk requirements.

3.4 Test Design

Each test case should include:

- test case ID;
- title;
- objective;
- linked requirement IDs;
- test type;
- priority or risk level;
- preconditions;
- test environment;
- input data;
- steps;
- expected results;
- acceptance criteria;
- evidence to capture;
- responsible role or assignee;
- estimated duration.

3.5 Test Execution

During execution, testers should record:

- actual result;
- pass/fail/blocked status;
- timestamps;
- environment version;
- data set used;
- logs, screenshots, or measurements;
- defect IDs;
- deviations from the planned procedure.

Blocked tests should not be counted as passed.

3.6 Reporting

A test report should summarize:

- test scope executed;
- tests passed, failed, blocked, and not run;
- requirement coverage;
- known gaps;
- defects by severity;
- residual risks;
- recommendation for release or further testing.

4. Test Plan Template

4.1 Header

Field	Description
Test plan ID	Unique identifier
Project	Project name
Product/version	Product or release under test
Author	Test plan owner
Reviewers	People who reviewed the plan
Date	Creation or update date
Status	Draft, reviewed, approved, obsolete

4.2 Introduction

Describe why the test plan exists and what decision it supports.

Example:

This test plan validates release candidate 1 of the gateway firmware before pilot deployment.

4.3 Objectives

Objectives should be outcome-oriented.

Good examples:

- verify that core telemetry flows operate under nominal and degraded network conditions;
- verify that security controls prevent unauthorized configuration changes;
- provide evidence for release readiness review.

Weak examples:

- test everything;
- check the system;
- validate the product.

4.4 Scope

Define what is included.

Example:

- API behavior;
- device registration workflow;
- local buffering;
- alert generation;
- security access control.

4.5 Out of Scope

Define what is excluded.

Example:

- mobile application;
- long-term cloud analytics;
- hardware certification;
- production scalability beyond the reference load.

4.6 Test Strategy

Describe how the test team will obtain evidence.

The strategy should explain:

- test levels;
- prioritization method;
- automation strategy;
- manual review points;
- regression approach;
- data management;
- defect handling.

4.7 Entry Criteria

Testing should start only when:

- requirements are baselined or stable enough;
- environment is available;
- test accounts are configured;
- test data is ready;
- critical dependencies are accessible.

4.8 Exit Criteria

Testing may finish when:

- all high-priority tests are executed or dispositioned;
- no open critical defects remain;
- requirement coverage is reviewed;
- residual risks are documented;
- stakeholders approve the result.

4.9 Test Cases

The plan should list test cases or link to a test case repository.

Each test case should be traceable to one or more requirements.

4.10 Coverage Matrix

The coverage matrix maps requirements to test cases.

Requirement ID	Requirement title	Test case IDs	Coverage status
REQ-001	User authentication	TC-001, TC-002	Covered
REQ-002	Password reset	TC-003	Covered
REQ-003	Audit export	none	Gap

5. Test Case Quality Checklist

A test case is ready for execution when:

- the objective is clear;
- linked requirements are listed;
- preconditions are explicit;
- steps are ordered;
- expected results are observable;
- acceptance criteria are measurable;
- required data and tools are available;
- evidence requirements are stated;
- pass/fail decision can be made without guessing.

6. Common Anti-Patterns

6.1 Test Case Without Requirement Link

Problem:

- creates untraceable effort;
- makes coverage reporting unreliable.

Correction:

- link the test to a requirement, risk, defect, or exploratory charter.

6.2 Expected Result Is Too Vague

Bad:

- system works correctly.

Better:

- API returns HTTP 201 and the created device appears in the inventory within 5 seconds.

6.3 Too Many Assertions in One Test

Problem:

- failure diagnosis becomes difficult.

Correction:

- split the test or clearly separate checkpoints.

6.4 Ignoring Negative Tests

Problem:

- confirms only happy paths.

Correction:

- include invalid input, boundary conditions, unauthorized users, network failure, and malformed data.

7. Guidance for AI-Generated Test Plans

When an AI system generates a test plan, reviewers should check:

- whether every test case has requirement links;
- whether invented requirements were added;
- whether source documents are cited;
- whether acceptance criteria are measurable;

- whether high-risk requirements are prioritized;
- whether the plan distinguishes functional, performance, security, and reliability testing;
- whether the output is executable by a real test team.

The AI should not claim full coverage unless the coverage matrix proves it.

8. References

- ISO/IEC/IEEE 29119 software testing standards overview and documentation templates.
- ISO/IEC/IEEE 29119-3: test documentation templates and examples.
- NASA Systems Engineering Handbook guidance on verification and validation evidence.